

Chapter 12: Binary Search Trees

1) Search trees are data structures that support many dynamic set operations including SEARCH, MINIMUM, MAXIMUM, PREDECESSOR, SUCCESSOR, INSERT and DELETE. Thus, a search tree can be used both as a dictionary and as a priority queue.

2) A binary search tree is a binary tree in which the keys of the node satisfy the binary-search-tree property:-

Let x be a node in the binary search tree. If y is a node in the left-subtree of x then $\text{Key}[y] \leq \text{Key}[x]$. If y is a node in the right subtree of x then $\text{Key}[y] \geq \text{Key}[x]$.

A binary search tree can be represented by a linked data structure in which each node is an object comprising of the following fields:-

- (i) Key
- (ii) Satellite data
- (iii) pointer to the left child
- (iv) pointer to the right child
- (v) pointer to the parent

If a child or parent is missing, the corresponding field contains the value NIL. Root is the only node whose parent field is NIL.

3) Inorder tree walk:-

The binary-search-tree property helps us to print out all the keys in a binary search tree in sorted order by using the inorder tree walk algorithm.

The inorder tree walk algorithm is called so because it prints the key of the root of a subtree between the keys in the left subtree and the keys in the right subtree.

Preorder tree walk prints the root before the values in either subtree.

Postorder tree walk prints the root after the values in its subtrees.

INORDER-TREE-WALK(x)

- 1 if $x \neq \text{NIL}$
- 2 then INORDER-TREE-WALK(left[x])
- 3 print Key[x]
- 4 INORDER-TREE-WALK(right[x])

→ It takes $\Theta(n)$ time to walk an n -node binary search tree.

Theorem: If x is the root of an n -node subtree, then the call $\text{INORDER-TREE-WALK}(x)$ takes $\Theta(n)$ time.

Let $T(n)$ be the time taken by INORDER-TREE-WALK . For an empty subtree INORDER-TREE-WALK takes a small, constant amount of time (for the first $x \neq \text{NIL}$). Let $T(0) = c$ for some +ve constant c .

Suppose INORDER-TREE-WALK is called on a node x whose left subtree has K nodes and whose right subtree has $n-K-1$ nodes.

Then,

$$T(n) = T(K) + T(n-K-1) + d$$

where, $d =$ time taken to execute $\text{INORDER-TREE-WALK}(x)$ - time spent in recursive calls

We use substitution method to show that $T(n) = \Theta(n)$ by proving that $T(n) = (c+d)n + c$.

For $n=0$, we have $(c+d) \cdot 0 + c = c = T(0)$.

For $n > 0$, we have

$$\begin{aligned} T(n) &= T(K) + T(n-K-1) + d \\ &= (c+d)K + c + (c+d)(n-K-1) + c + d \\ &= (c+d)n + c \quad \text{which completes the proof.} \end{aligned}$$

4) Querying a binary search tree:-

A binary search tree supports the querying operations of SEARCH, MINIMUM, MAXIMUM, SUCCESSOR and PREDECESSOR.

1) SEARCHING:

Given a pointer x to the root of the binary search tree and a Key K , TREE-SEARCH returns a pointer to a node with Key K if one exists; otherwise it returns NIL.

TREE-SEARCH(x, K)

- 1 if $x = \text{NIL}$ or $K = \text{Key}[x]$
- 2 then return x
- 3 if $K < \text{Key}[x]$
- 4 then return TREE-SEARCH(left[x], K)
- 5 else return TREE-SEARCH(right[x], K)

→ The running time of TREE-SEARCH is $O(h)$, where h is the height of the tree, since the nodes encountered during recursion form a path downward from the root of the tree. (The maximum length of a path from the root to any leaf is equal to the height of the tree).

→ If we convert the recursion into a while loop we have the following iterative version of TREE-SEARCH:

ITERATIVE-TREE-SEARCH(x, K)

```
1 while  $x \neq \text{NIL}$  and  $K \neq \text{Key}[x]$ 
2     do if  $K < \text{Key}[x]$ 
3         then  $x \leftarrow \text{left}[x]$ 
4         else  $x \leftarrow \text{right}[x]$ 
5 return  $x$ 
```

2) MINIMUM: The minimum key in a binary search tree can be found by following the left pointers from the root.

TREE-MINIMUM(x)

```
1 while  $\text{left}[x] \neq \text{NIL}$ 
2     do  $x \leftarrow \text{left}[x]$ 
3 return  $x$ 
```

→ The TREE-MINIMUM procedure runs in $O(h)$ time on a tree of height h , since a the sequence of nodes encountered forms a path downward from the root.

3) MAXIMUM: The maximum key in a binary search tree can be found by following the right pointers from the root.

↳ TREE-MAXIMUM(x)

- 1 while $\text{right}[x] \neq \text{NIL}$
- 2 do $x \leftarrow \text{right}[x]$
- 3 return x

→ The TREE-MAXIMUM procedure runs in $O(h)$ time on a tree of height h since the sequence of nodes encountered forms a path downward from the root.

4) SUCCESSOR :

The successor of a node x in the binary search tree is the node with the smallest key greater than $\text{key}[x]$. The structure of a binary search tree allows us to determine the successor of a node without ever comparing the keys.

TREE-SUCCESSOR(x)

- 1 if $\text{right}[x] \neq \text{NIL}$
- 2 then return TREE-MINIMUM($\text{right}[x]$)
- 3 $y \leftarrow \text{parent}[x]$
- 4 while $y \neq \text{NIL}$ and $x = \text{right}[y]$
- 5 do $x \leftarrow y$
- 6 $y \leftarrow \text{parent}[y]$

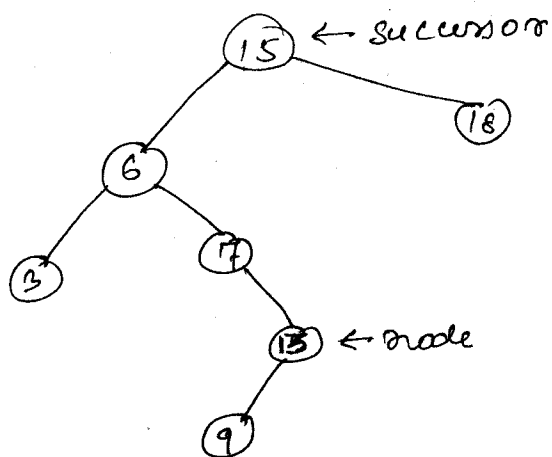
return y

The pseudocode for TREE-SUCCESSOR is broken into two cases:-

(i) If the right subtree of x is non-empty then the successor of x is just the leftmost node in the right subtree.

(ii) If the right subtree of node x is empty and x has a successor y , then y is the lowest ancestor of x whose left child is also an ancestor of x . To find y , we simply go up the tree from x until we encounter a node that is the left child of its parent.

eg:-



→ The running time of TREE-SUCCESSOR on a tree of height h is $O(h)$, since we either follow a path up the tree or follow a path down the tree.

→ **Theorem:** All the querying operations can be performed in time $O(h)$ on a binary search tree of height h .

5) Insertion into a binary search

tree: The tree insert procedure is passed a pointer to the node to be inserted into the tree.

TREE-INSERT(T, z)

```
1  trav ← root[T]
2  parent-trav ← NIL
3  while trav ≠ NIL
4      do parent-trav ← trav
5          if Key[z] < Key[trav]
6              then trav ← left[trav]
7              else trav ← right[trav]
8  parent[z] ← parent-trav
9  if parent-trav = NIL    ▷ tree T was empty
10     then root[T] ← z
11     else if Key[z] < Key[parent-trav]
12         then left[parent-trav] ← z
13         else right[parent-trav] ← z
```


→ Tree insert takes $O(h)$ time for a tree of height h .

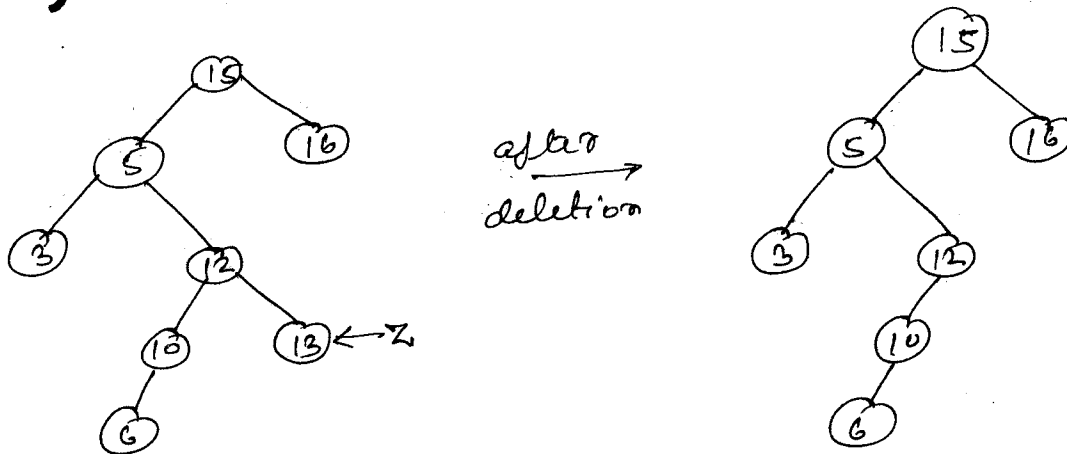
6) Deletion from a binary search tree:

Theorem: If a node in a binary search tree has two children, then its successor has no left child and its predecessor has no right child.

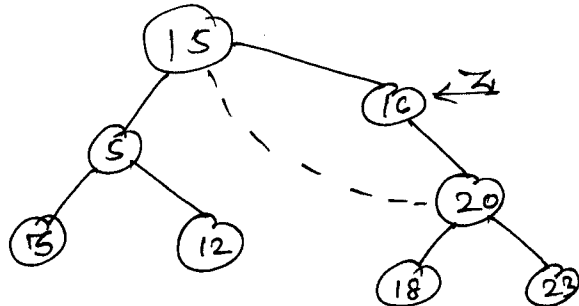
3 cases arise in case of deletion:

Which node is actually removed depends upon how many children Z has.

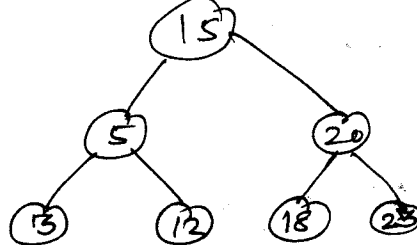
a) If Z has no children: we just remove it.



b) If Z has only one child: we splice out Z by making a link between its parent and its only child

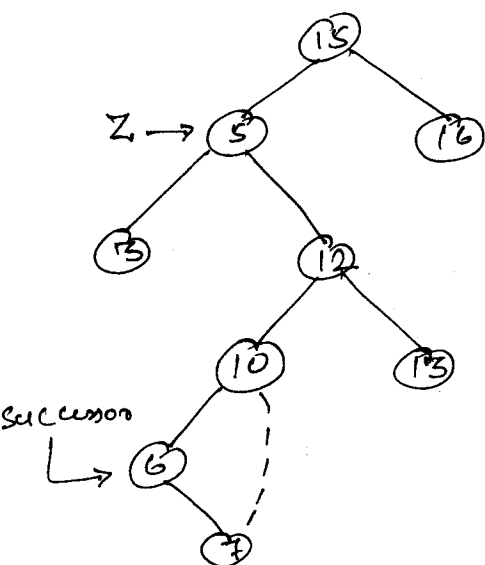


after
deletion

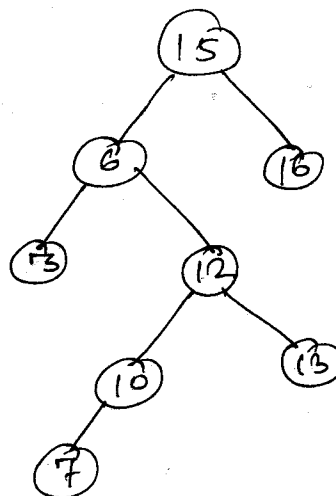


c) if Z has 2 children: we splice out

Z 's successor which has no left child
(and may or may not have right child)
and replace Z 's key and satellite data
with the successor's key and satellite data



after
deletion



The code for TREE_DELETE organizes these
cases a little differently:-

TREE_DELETE(T, Z)

1 if $\text{left}[Z] = \text{NIL}$ or $\text{right}[Z] = \text{NIL}$

2 then splice-node $\leftarrow z$
3 else splice-node $\leftarrow \text{TREE-SUCCESSOR}[z]$

▷ first determine the node to be spliced

4

▷ the node to be spliced can have only a single child, determine it

4 if left[splice-node] $\neq \text{NIL}$

5 then splice-child $\leftarrow \text{left}[\text{splice-node}]$

6 else splice-child $\leftarrow \text{right}[\text{splice-node}]$

7 if splice-child $\neq \text{NIL}$

8 then parent[splice-child] $\leftarrow \text{parent}[\text{splice-node}]$

9 if parent[splice-node] $= \text{NIL}$

10 then root[T] $\leftarrow \text{splice-child}$

11 else if splice-node $= \text{left}[\text{parent}[\text{splice-node}]]$

12 then left[parent[splice-node]]
 $\leftarrow \text{splice-child}$

13 else right[parent[splice-node]]
 $\leftarrow \text{splice-child}$

14 if splice-node $\neq z$

15 then Key[z] $\leftarrow \text{Key}[\text{splice-node}]$

16 Copy splice-node's satellite data

17 return splice-node into z //

→ The procedure TREE-DELETE runs in $O(h)$ time on a tree of height h .

→ **Theorem:** The dynamic set operations INSERT and DELETE can be made to run in $O(h)$ time on a binary search tree of height h .

7) Randomly built binary search trees:

All the basic operations on a binary search tree run in $O(h)$ time where h is the height of the tree. The height of a binary search tree varies, however, as items are inserted and deleted. If, for example, the items are inserted in strictly increasing order, the tree will be a chain with height $n-1$.

On the other hand $h \geq \lceil \lg n \rceil$. The behaviour of the average case is much closer to the best case than the worst case. Little is known about the average height of a binary search tree when both insertion and deletion are used to create it. When the tree is created by insertion alone the analysis becomes more tractable.

A randomly built binary search tree on n keys is defined as a binary search tree that arises from inserting the keys

random order into an initially empty tree, where each of the $L!$ permutations of the input keys is equally likely.

The expected height of a randomly built binary search tree ^{on n keys} is $O(\lg n)$, so that basic dynamic-set operations on such a tree take $\Theta(\lg n)$ time on average.